DirtyDecrypt

Linux Kernel Local Privilege Escalation — CVE-2026-31635

Threat Intelligence Report | Published: 15 May 2026 | Lost Boys Cyber

Attribute	Detail
Vulnerability Name	DirtyDecrypt (alias: DirtyCBC)
CVE ID	CVE-2026-31635
CVSSv3.1 Score	7.5 — High | Vector: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-787: Out-of-Bounds Write (page-cache corruption via missing COW guard)
Vulnerability Type	Local Privilege Escalation (LPE) — unprivileged user to root
Affected Component	Linux kernel rxgk_decrypt_skb() — RxGK subsystem (net/rxrpc/rxgk*.c)
Kernel Config Req.	CONFIG_RXGK must be compiled in — limits scope to specific distros
Affected Distros	Fedora, Arch Linux, openSUSE Tumbleweed — NOT Ubuntu, RHEL, Debian
Patch Date	25 April 2026 — upstream mainline kernel (quietly merged)
Public Disclosure	~14 May 2026 — V12 Security Team independent discovery and PoC release
Exploit Status	Public PoC available — GitHub: v12-security/pocs/dirtydecrypt
Attack Vector	Local — requires existing low-privilege user account on target system
Authentication	Low-privileged local user account required
No Interaction	User interaction not required — fully local exploit

AI-ASSISTED REPORT — IMPORTANT DISCLOSURE

AI Generation: This report was generated with AI assistance from publicly available sources. All content, findings, detection rules, and recommendations must be independently reviewed by a qualified security professional before being acted upon or implemented in any environment.

Query Disclaimer: All detection queries (Sigma, bash, auditd rules) are generic templates requiring adaptation to your specific environment, kernel version, log sources, and SIEM/EDR configuration.

Currency: Based on publicly available sources as of 15 May 2026. Kernel LPE exploitation techniques evolve rapidly — verify all details against the upstream kernel commit, NVD entry, and your distribution's security advisory before relying on this document.

1. Executive Summary

DirtyDecrypt (CVE-2026-31635), also known as DirtyCBC, is a high-severity local privilege escalation vulnerability in the Linux kernel that allows an unprivileged local user to escalate to root by exploiting a missing Copy-On-Write (COW) guard in the rxgk_decrypt_skb() function within the kernel's RxGK subsystem. The upstream kernel patch was quietly merged on 25 April 2026. The V12 Security Team independently discovered and publicly disclosed the vulnerability — along with a working proof-of-concept exploit — in mid-May 2026, dramatically lowering the barrier to exploitation for unpatched systems.

The RxGK subsystem provides the GSS-API-based security layer for RxRPC, the network transport underlying the Andrew File System (AFS) client in Linux. Because RxGK support requires the

CONFIG_RXGK kernel configuration option, the vulnerability's scope is naturally limited to distributions that track the upstream kernel closely and compile this option in — primarily Fedora, Arch Linux, and openSUSE Tumbleweed. Mainstream enterprise distributions including Ubuntu, Red Hat Enterprise Linux, Debian, and Amazon Linux are not affected in their stock kernel configurations.

The exploitation mechanism is elegant and deterministic: by sending an oversized response authenticator to the rxgk_decrypt_skb() function — bypassing the absent COW check — an attacker can write arbitrary data into the kernel's page cache at locations owned by privileged files, including SUID root binaries. A poisoned SUID binary, when subsequently executed by any user, provides a root shell. Unlike race-based LPE techniques (e.g., the original DirtyPipe), DirtyDecrypt does not require timing precision and is therefore highly reliable in practice.

Organisations running affected distributions should immediately apply the latest kernel updates. Where patching is not immediately possible, the rxrpc kernel module can be blacklisted as a temporary mitigation, though this disables AFS client functionality. Container environments running affected host kernels are also at risk — a containerised process that obtains local code execution on the host can leverage DirtyDecrypt to escape the container entirely.

2. Vulnerability Details

2.1 Background — RxGK and the AFS Subsystem

RxRPC is a reliable, connection-oriented datagram protocol used by the kernel's AFS (Andrew File System) client to communicate with AFS file servers. RxGK (RxRPC Generic Key) is the GSS-API-based security layer built on top of RxRPC, providing authentication, confidentiality, and integrity protection for AFS communications. The kernel's rxgk module (net/rxrpc/rxgk*.c) implements this security layer, including the rxgk_decrypt_skb() function responsible for decrypting incoming network packets.

Because AFS is a niche distributed filesystem primarily used in academic and enterprise high-performance computing environments, and because CONFIG_RXGK is not enabled in the default kernel builds of major enterprise distributions, this subsystem has historically received less security scrutiny than more widely-used kernel components. This makes it a productive surface for vulnerability research.

2.2 Root Cause — Missing COW Guard

The core vulnerability is a missing Copy-On-Write (COW) guard in the rxgk_decrypt_skb() function. In the Linux kernel, COW semantics ensure that when a kernel operation needs to modify shared memory pages — such as those in the page cache — a private copy is made before modification, preventing unintended writes to the original shared page.

In rxgk_decrypt_skb(), the function decrypts incoming RxGK-protected network packets (sk_buff structures). When processing response authenticators, the function fails to verify the COW state of the page before writing decrypted data into it. If the target page is a shared page-cache page — for example, a page belonging to a memory-mapped SUID binary — the function writes directly to the shared page rather than creating a private copy.

Additionally, the function does not enforce an upper bound on the size of incoming response authenticators. An attacker can craft an oversized authenticator that, when 'decrypted', writes attacker-controlled data beyond the legitimate buffer region, further extending the write primitive into adjacent page-cache pages. The combination of the missing COW guard and the missing length validation creates a powerful, deterministic write primitive into kernel page cache.

2.3 Exploitation Mechanics

The V12 Security Team's published PoC demonstrates the following exploitation chain:

- Verify CONFIG_RXGK is active: grep CONFIG_RXGK /boot/config-$(uname -r) — must return '=y' or '=m'.
- Load the rxrpc kernel module if not already loaded: modprobe rxrpc (standard user can do this if CONFIG_RXGK=m).
- Identify a target SUID root binary (e.g. /usr/bin/sudo or /usr/bin/passwd) whose pages are present in the kernel page cache.
- Craft a malicious RxGK response authenticator containing a shellcode payload sized to overwrite the target function in the SUID binary's page-cache representation.
- Send the crafted authenticator to trigger rxgk_decrypt_skb() — writing the payload into the SUID binary's page-cache pages without triggering COW.
- Execute the now-poisoned SUID binary. The kernel executes the modified page-cache pages — which contain attacker shellcode — with the SUID binary's effective UID (root).
- A root shell is obtained. The page-cache modification is non-persistent (survives until the page is evicted or the system reboots).

Unlike DirtyPipe (CVE-2022-0847), which required a race condition between threads, DirtyDecrypt is a deterministic logical flaw. Exploitation is straightforward and reliable. The non-persistent nature of the page-cache modification also makes forensic recovery of the attack difficult — there is no on-disk evidence of the modification after the affected page is evicted.

2.4 Affected Kernel Versions and Distributions

The vulnerability affects Linux kernels with CONFIG_RXGK compiled in, prior to the upstream patch merged on 25 April 2026. The following distributions are confirmed affected in their default kernel configurations:

- Fedora 41 / 42 — ships with CONFIG_RXGK=m; affected kernel versions prior to the post-April 25 update
- Arch Linux — rolling release; affected until kernel package update post April 25, 2026
- openSUSE Tumbleweed — rolling release; affected until kernel update incorporating the patch

The following major distributions are NOT affected in their stock configurations:

- Ubuntu (all supported releases) — CONFIG_RXGK not enabled
- Red Hat Enterprise Linux / CentOS Stream / AlmaLinux / Rocky Linux — CONFIG_RXGK not enabled
- Debian (all branches including sid) — CONFIG_RXGK not enabled
- Amazon Linux 2 / 2023 — CONFIG_RXGK not enabled
- SUSE Linux Enterprise Server (SLES) — CONFIG_RXGK not enabled in SLES kernels (distinct from Tumbleweed)

2.5 Container and Cloud Exposure

Kubernetes and container environments running affected host kernels are also at risk. Container namespacing does not protect against kernel LPE vulnerabilities — a containerised process that achieves

code execution on the host kernel can leverage DirtyDecrypt to escalate to root on the underlying host, achieving full container escape. All worker nodes in a Kubernetes cluster running an affected Fedora, Arch, or openSUSE kernel should be considered at risk until patched.

2.6 Relationship to the 'Dirty' LPE Vulnerability Family

DirtyDecrypt belongs to a broader class of Linux kernel page-cache corruption vulnerabilities that have been disclosed in quick succession throughout 2025-2026. This family includes:

- DirtyPipe (CVE-2022-0847) — pipe buffer flag handling race, enabling page-cache overwrite of SUID binaries. Patched in kernel 5.16.11 / 5.15.25.
- Dirty Frag (CVE-2026-43284, CVE-2026-43500) — deterministic LPE via ESP (IPsec) and RxRPC fragmentation logic flaws. Affects all major distributions.
- Fragnesia (CVE-2026-46300) — LPE spawned by the Dirty Frag patch itself; logic error in the remediation code created a new exploitable condition.
- DirtyDecrypt / DirtyCBC (CVE-2026-31635) — missing COW guard in rxgk_decrypt_skb, page-cache corruption via oversized AFS authenticators.

The pattern of repeated page-cache write primitives emerging from Linux kernel networking subsystems suggests a systemic gap in security review coverage for less-prominent protocol implementations — subsystems like RxRPC and AFS that are correct in common cases but have not received the same hardening scrutiny as core networking stacks.

3. Threat Landscape & Context

DirtyDecrypt was patched upstream without public fanfare on 25 April 2026, approximately three weeks before the V12 Security Team independently rediscovered the vulnerability and published both the technical analysis and a working PoC exploit in mid-May 2026. The upstream kernel commit message did not reference the security impact — a practice common for sensitive fixes that are intended to land in rolling-release distributions before the vulnerability is widely known.

The publication of the V12 PoC on GitHub (v12-security/pocs) fundamentally changes the threat landscape. Prior to the PoC release, exploitation required a researcher-level understanding of kernel page-cache internals. Post-publication, the barrier to exploitation is substantially lower — motivated script-level attackers can adapt and execute the PoC with minimal kernel expertise. Given that Fedora is widely used in developer workstations and CI/CD runner infrastructure, and that Arch Linux is common in security-research and DevOps environments, the realistic attacker population for DirtyDecrypt is broader than a niche LPE might suggest.

The primary exploitation scenario is a threat actor with initial foothold as a low-privilege user — obtained via phishing, credential stuffing, supply-chain compromise, or exploitation of a web application — who uses DirtyDecrypt to escalate to root. From root, the attacker can disable security tooling, dump /etc/shadow for offline cracking, install kernel rootkits or backdoors, pivot to adjacent network infrastructure, and exfiltrate data. In a Kubernetes context, full container escape to the host node enables cluster-wide compromise.

No threat actor attribution has been made for active DirtyDecrypt exploitation at the time of this report. However, LPE vulnerabilities with public PoCs in this CVSS range (7.5+) typically see integration into post-exploitation frameworks (Metasploit, Cobalt Strike loaders) within 2–4 weeks of PoC publication, and integration into automated exploitation tooling used by ransomware affiliates within 4–8 weeks. Organisations should treat the patching window as critically short.

4. Mitigation

4.1 Apply Kernel Updates (Primary Mitigation)

The definitive fix is to update to a kernel version that includes the upstream patch merged on 25 April 2026. Update commands by distribution:

Fedora:

```
sudo dnf upgrade --refresh kernel
sudo reboot
uname -r  # Verify running kernel after reboot
```

Arch Linux:

```
sudo pacman -Syu linux linux-headers
sudo reboot
```

openSUSE Tumbleweed:

```
sudo zypper dup
sudo reboot
```

4.2 Verify CONFIG_RXGK Status

Before determining if a system is affected, verify whether CONFIG_RXGK is active in the running kernel:

```
grep CONFIG_RXGK /boot/config-$(uname -r)
# CONFIG_RXGK=y  → built-in, affected if unpatched
# CONFIG_RXGK=m  → loadable module, affected if module is loaded or can be loaded
# CONFIG_RXGK is not set → NOT affected

# Check if module is currently loaded:
lsmod | grep rxrpc
```

4.3 Module Blacklist — Interim Mitigation

Where immediate kernel patching is not possible, blacklisting the rxrpc kernel module prevents the vulnerability from being triggered. WARNING: this also disables the AFS client (afs.ko) and any services depending on it. Evaluate the operational impact before deploying.

```
# Create blacklist entry and unload module
echo 'install rxrpc /bin/false' | sudo tee /etc/modprobe.d/dirtydecrypt.conf
sudo rmmod rxrpc 2>/dev/null || true
sudo dracut -f  # Rebuild initramfs to apply blacklist at boot (Fedora/RHEL)
# OR on Arch: sudo mkinitcpio -P

# Verify module is blocked:
sudo modprobe rxrpc && echo 'BLOCKED FAILED' || echo 'Blocked successfully'
```

To restore AFS functionality after patching, remove the blacklist entry and regenerate the initramfs:

```
sudo rm /etc/modprobe.d/dirtydecrypt.conf
sudo dracut -f  # Or mkinitcpio -P on Arch
```

4.4 Harden SUID Binary Attack Surface

The exploitation technique requires writing to the page cache of an SUID root binary. Auditing and minimising the SUID binary footprint reduces the number of viable exploitation targets, and enables more targeted monitoring:

```
# Enumerate all SUID binaries on the system
find / -perm -4000 -type f 2>/dev/null | sort

# Remove unnecessary SUID bits — example:
sudo chmod u-s /usr/bin/at         # If at(1) is not needed
sudo chmod u-s /usr/bin/newgrp     # If multi-group switching not required
```

Enable systemd's ProtectSystem and NoNewPrivileges for services that do not require privilege escalation — this prevents SUID binaries from being useful as escalation vectors even if page cache corruption occurs within a sandboxed process context.

5. Detection

5.1 Detection Strategy

DirtyDecrypt's page-cache corruption approach is inherently stealthy — the modification is in-memory only, leaves no on-disk trace, and the kernel's normal security auditing paths are not traversed during the rxgk_decrypt_skb write. Detection therefore relies on behavioural indicators: anomalous use of the rxrpc/rxgk kernel subsystem, unusual process privilege transitions, and unexpected child processes spawned by SUID binaries.

5.2 Check Exposure — Rapid Assessment Script

Deploy this script to rapidly assess whether a Linux host is vulnerable to DirtyDecrypt:

```
#!/bin/bash
# DirtyDecrypt (CVE-2026-31635) Exposure Check — Lost Boys Cyber
KERNEL=$(uname -r)
CONFIG=/boot/config-${KERNEL}

echo "[*] Kernel: ${KERNEL}"

# Check CONFIG_RXGK
if [ -f "${CONFIG}" ]; then
  RXGK=$(grep CONFIG_RXGK ${CONFIG} | head -1)
  echo "[*] CONFIG_RXGK: ${RXGK}"
  if echo "${RXGK}" | grep -qE '=(y|m)'; then
    echo "[!] POTENTIALLY AFFECTED — CONFIG_RXGK is enabled"
  else
    echo "[+] NOT AFFECTED — CONFIG_RXGK not enabled"
    exit 0
  fi
fi
```

```
# Check if rxrpc module is loaded
if lsmod 2>/dev/null | grep -q rxrpc; then
  echo "[!] rxrpc MODULE IS LOADED — actively exploitable"
else
  echo "[~] rxrpc not currently loaded (module-form only)"
fi

# Check for blacklist mitigation
if grep -r 'install rxrpc /bin/false' /etc/modprobe.d/ 2>/dev/null | grep -q .;
then
  echo "[+] MITIGATED — rxrpc blacklisted in modprobe.d"
else
  echo "[!] No blacklist mitigation found"
fi
```

5.3 Sigma Rule — rxrpc Module Load by Unprivileged User

```
title: DirtyDecrypt — rxrpc Module Loaded by Non-Root User
id: f3a19c82-4d71-4e2b-b301-9a7e22c4d051
status: experimental
description: |
  Detects non-root users loading the rxrpc kernel module, which is a required
  step for DirtyDecrypt (CVE-2026-31635) exploitation when CONFIG_RXGK=m.
  Legitimate AFS usage of rxrpc is typically loaded at system boot by root.
author: Lost Boys Cyber
date: 2026-05-15
logsource:
  category: process_creation
  product: linux
detection:
  selection:
    Image|endswith:
      - '/modprobe'
      - '/insmod'
    CommandLine|contains:
      - 'rxrpc'
      - 'rxgk'
  filter_root:
    User: 'root'
  condition: selection and not filter_root
falsepositives:
  - AFS administrators loading rxrpc during maintenance (validate against change
records)
level: high
tags:
  - attack.privilege_escalation
  - attack.t1068
  - cve.2026-31635
```

5.4 Sigma Rule — SUID Binary Spawning Unexpected Shell

```
title: DirtyDecrypt — SUID Binary Spawns Interactive Shell
id: a8c74b23-5f82-4d91-c702-8b3e33d5e062
status: experimental
description: |
  Detects common SUID binaries (sudo, passwd, su, newgrp) spawning interactive
  shells or interpreters — a pattern consistent with page-cache poisoning via
  DirtyDecrypt or similar LPE techniques. Adjust ParentImage list to your
  environment's SUID binary inventory.
author: Lost Boys Cyber
date: 2026-05-15
logsource:
  category: process_creation
  product: linux
detection:
  selection_parent:
    ParentImage|endswith:
      - '/sudo'
      - '/passwd'
      - '/su'
      - '/newgrp'
      - '/ping'
      - '/mount'
  selection_child:
    Image|endswith:
      - '/bash'
      - '/sh'
      - '/dash'
      - '/zsh'
      - '/python'
      - '/python3'
      - '/perl'
  condition: selection_parent and selection_child
falsepositives:
  - sudo legitimately spawning shells (e.g. sudo bash) — tune with CommandLine
context
level: high
tags:
  - attack.privilege_escalation
  - attack.t1068
```

5.5 Auditd Rules — Kernel LPE Detection

Configure Linux auditd to capture key events associated with DirtyDecrypt exploitation and post-escalation activity:

```
# /etc/audit/rules.d/dirtydecrypt.rules
# Monitor modprobe/insmod invocations for rxrpc
-a always,exit -F arch=b64 -S execve -F path=/usr/bin/modprobe \
  -F auid>=1000 -F auid!=4294967295 -k rxrpc_load

# Monitor SUID binary execution by non-root users
-a always,exit -F arch=b64 -S execve -F euid=0 \
  -F auid>=1000 -F auid!=4294967295 -k suid_exec

# Monitor /etc/shadow and /etc/passwd writes (post-escalation persistence)
-w /etc/shadow -p wa -k sensitive_file_write
-w /etc/passwd -p wa -k sensitive_file_write
-w /etc/sudoers -p wa -k sudoers_write

# Monitor cron/systemd service creation (post-root persistence)
-w /etc/cron.d -p wa -k cron_persistence
-w /etc/systemd/system -p wa -k systemd_persistence

# Reload rules: augenrules --load
```

6. Threat Hunting

6.1 Hunt: Systems with rxrpc Loaded and Unpatched Kernels

Deploy the exposure check script from Section 5.2 fleet-wide via your configuration management platform (Ansible, SaltStack, Puppet, or cloud-init). Identify all hosts matching both conditions: CONFIG_RXGK enabled AND kernel version prior to the post-April 25 patch.

```
# Ansible ad-hoc — check kernel version and RXGK status
ansible all -m shell -a \
  'echo "$(uname -r): $(grep CONFIG_RXGK /boot/config-$(uname -r) || echo
NOT_SET)"'
# Pipe output to grep '=y\|=m' to identify affected hosts
```

6.2 Hunt: Unexpected Privilege Transitions (Root Processes from User Sessions)

Use auditd EXECVE records with euid=0 and auid>=1000 to identify privilege transitions that didn't originate from expected sudo/su invocations. Correlate with the timing of known DirtyDecrypt PoC activity windows.

```
# Query auditd logs for SUID execution (suid_exec key)
ausearch -k suid_exec --start today -i | \
  awk '/type=EXECVE/{cmd=$0} /uid=[^0].*euid=0/{print cmd}' | \
  grep -v 'sudo\|su\|passwd\|login'

# Look for unexpected root shells
```

```
ausearch -k suid_exec --start today -i | \
  grep -E '(bash|sh|zsh|python)[^a-z]' | head -50
```

6.3 Hunt: Post-Exploitation Persistence Indicators

Following a successful DirtyDecrypt escalation, threat actors commonly establish persistence. Hunt for the following post-root indicators:

- /etc/shadow or /etc/passwd modification outside of expected provisioning windows (auditd key: sensitive_file_write)
- New entries in /etc/sudoers or /etc/sudoers.d/ granting passwordless sudo to unexpected users
- New cron jobs in /etc/cron.d/, /var/spool/cron/crontabs/, or additions to root's crontab
- New systemd unit files in /etc/systemd/system/ or /lib/systemd/system/ not present in baseline
- New SSH authorized_keys entries in /root/.ssh/authorized_keys
- New SUID binaries appearing on the system (compare against your SUID baseline from Section 4.4)
- Unexpected kernel modules loaded — lsmod output compared against a known-good baseline

6.4 Hunt: Container Escape Indicators

In Kubernetes or container environments, hunt for process trees that originate within container namespaces but achieve root on the host. Key indicators include processes with effective UID 0 whose namespace ancestry includes a container runtime (containerd, crun, runc) but whose filesystem access patterns include host paths outside the container's mount namespace.

```
# List all root processes and their namespace membership
ps aux | awk '$1=="root" {print $2}' | \
  xargs -I{} sh -c 'echo -n "{}: "; cat /proc/{}/cgroup 2>/dev/null | head -1'

# Flag root processes inside container cgroups (unexpected)
ps aux | awk '$1=="root"{print $2}' | \
  xargs -I{} sh -c \
  'grep -l docker /proc/{}/cgroup 2>/dev/null && echo "CONTAINER ROOT PID: {}"'
```

7. MITRE ATT&CK Mapping

Tactic	ID	Technique	Application
Privilege Escalation	T1068	Exploitation for Privilege Escalation	DirtyDecrypt: Exploits missing COW guard in rxgk_decrypt_skb() to write to SUID binary page cache and obtain a root shell.
Defense Evasion	T1014	Rootkit	Post-exploitation: Attacker with root access may install a kernel rootkit (LKM) to persist and hide presence.
Defense Evasion	T1055	Process Injection	Exploitation technique: Overwrites memory of a SUID root process (page-cache) to inject attacker shellcode.

Defense Evasion	T1562.001	Impair Defenses: Disable or Modify Tools	Post-exploitation: Root access allows disabling auditd, SELinux, AppArmor, or EDR kernel modules.
Credential Access	T1003.008	OS Credential Dumping: /etc/passwd and /etc/shadow	Post-exploitation: Root access enables reading /etc/shadow for offline password cracking.
Persistence	T1098.004	Account Manipulation: SSH Authorized Keys	Post-exploitation: Attacker injects SSH key into /root/.ssh/authorized_keys for persistent root access.
Persistence	T1543.002	Create or Modify System Process: Systemd Service	Post-exploitation: New systemd service installed for persistence that survives reboots.
Execution	T1059.004	Command and Scripting Interpreter: Unix Shell	Post-exploitation: Root shell obtained via poisoned SUID binary execution — attacker executes arbitrary commands.
Lateral Movement	T1021.004	Remote Services: SSH	Post-exploitation: Root SSH keys or /etc/shadow credential dumps enable lateral movement to adjacent systems.
Impact	T1611	Escape to Host	Container context: LPE via DirtyDecrypt on host kernel enables escape from container namespace to host root.

8. Recommended Actions Summary

Patch is available. The window before widespread exploitation of the public PoC is narrow — treat this as P1 for all affected systems.

Action	Owner	Timeframe
Run exposure check script (Section 5.2) on all Linux hosts fleet-wide	SOC / IT Ops	Immediate
Apply kernel updates on all Fedora / Arch / openSUSE Tumbleweed systems	Patch Management	Immediate (< 24 hrs)
Blacklist rxrpc module on unpatched hosts where AFS is not required (Section 4.3)	IT Ops / Endpoint	Immediate (interim)
Audit and minimise SUID binary count across all affected Linux hosts (Section 4.4)	Endpoint Security	24-48 hours
Deploy auditd rules from Section 5.5 on all Linux endpoints	SOC / Detection Eng.	24-48 hours
Deploy Sigma rules for rxrpc module load and SUID shell spawning to SIEM	Detection Eng.	24-48 hours
Review Kubernetes worker node kernels — patch or isolate affected nodes	Platform / DevOps	48 hours
Review container security policies — enforce seccomp and AppArmor/SELinux profiles to limit LPE blast radius	Platform Security	48-72 hours
Hunt for post-exploitation indicators on all affected systems (Section 6)	Threat Intel / IR	48-72 hours
Confirm kernel update status via compliance scan — close loop on patching	GRC / IT Ops	72 hours

9. References

Bleeping Computer — 'Exploit available for new DirtyDecrypt Linux root escalation flaw': https://www.bleepingcomputer.com/news/security/exploit-available-for-new-dirtydecrypt-linux-root-escalation-flaw/

SecurityWeek — 'PoC Released for DirtyDecrypt Linux Kernel Vulnerability': https://www.securityweek.com/poc-released-for-dirtydecrypt-linux-kernel-vulnerability/

Aviatrix Threat Research — 'DirtyDecrypt Linux Kernel Vulnerability (CVE-2026-31635)': https://aviatrix.ai/threat-research-center/exploit-available-for-new-dirtydecrypt-linux-root-escalation-flaw/

Moselwal — 'DirtyDecrypt — Linux kernel LPE (CVE-2026-31635)': https://moselwal.com/blog/dirtydecrypt-linux-kernel-rxgk-cve-2026-31635

NVD — CVE-2026-31635 Detail: https://nvd.nist.gov/vuln/detail/CVE-2026-31635

V12 Security Team PoC Repository — GitHub: https://github.com/v12-security/pocs/tree/main/dirtydecrypt

Ubuntu Security — CVE-2026-31635 Advisory: https://ubuntu.com/security/CVE-2026-31635

PrivacyRadar — 'DirtyDecrypt Linux Security Bug Allows Local Root Privilege Escalation': https://privacyradar.com/news/cybersecurity/dirtydecrypt-linux-kernel-flaw-local-root-access/

Help Net Security — 'Fragnesia: New Linux kernel LPE bug was spawned by Dirty Frag patch (CVE-2026-46300)': https://www.helpnetsecurity.com/2026/05/14/fragnesia-cve-2026-46300-linux-lpe-vulnerability/

Sysdig — 'Dirty Frag (CVE-2026-43284 and CVE-2026-43500): Detecting unpatched LPE via Linux Kernel ESP and RxRPC': https://www.sysdig.com/blog/dirty-frag-cve-2026-43284-and-cve-2026-43500-detecting-unpatched-local-privilege-escalation-via-linux-kernel-esp-and-rxrpc

---

This report was compiled from publicly available threat intelligence as of 15 May 2026. A kernel patch is available — apply immediately on all affected systems (Fedora, Arch Linux, openSUSE Tumbleweed). Ubuntu, RHEL, and Debian are NOT affected in stock configurations. All detection content requires environment-specific validation before deployment.